Final project (E.L.A.S.)

Educational Log Analysis System (E.L.A.S.)

The project involves creating a system for processing and cleaning educational logs. As part of the assignment, you must provide a program that simulates the cleaning and processing of corrupted data. The application's configuration depends on a provided configuration file containing information about the cleaning stages and data modifications to be executed. The configuration should be a text file that is easy to edit manually, e.g., `.txt`, `.json`, `.yaml`, `.toml`. The author may choose **one** supported configuration file format.

A dataset for each student has been generated individually and is located in the MS-Teams directory in a file with a `.usrlog` extension. This file contains hundreds of irregularly structured rows, although the order of elements in each row is constant. The data is "dirty", meaning there may be situations where parts of the information are missing, the recorded data contains incorrectly encoded characters or noise, alternative key aliases were used, or incorrect separators are present.

Example of an input file row:

Output

```
studEnT=jEnNIFEr    heRnANdEz-GoNzaLez    ##    iD-REQ-205725    ##    addr-
Thomasstad,Carrie    Pass,90705    ##    resuLts-4.5Q,2.0&,2.0&,4.5&    ##
<phone:786-861-3914|email:EubFzT@gMaIL.coM>
```

```
sTuDenT-erIcA  ALExanDER  !!  iD-REQ-741419  !!  addr=Port  Robert,Michael
Way,22457  !!  RESuLTs:  2.0  ,4.0@,5.0@,  5.0   !!  <phone:+৯ ৭৭৫-৯২৪-৬৭৭৯|
email:xEKF@NyU.EDU>
```

To facilitate the task, the first 5 rows of the file present all types of "corruptions" and data formatting variants.

Configuration file examples:

pipeline_config.txt

```
[FIX]CHECK_TYPE   →   CHECK_REQUIRED(id,email)   →   [FIX]NORMALIZE   -
>   [FIX]GRADE_MAPPER   →   DEDUPLICATE   →   [FIX]CHECKSUM   →
SORT(grades_avg:desc,name:asc) → EXPORT_JSON
```

pipeline_config.json

```
[
  {"name": "CHECK_TYPE", "repair": true},
  {"name": "CHECK_REQUIRED", "args": ["id", "email"], "repair": false},
```

```
  {"name": "NORMALIZE", "repair": true}
  ....
]
```

Stage 1: Reading and Initial Mapping

In this stage, you must provide the necessary function/logic to read the data file line by line (to minimize RAM usage) using the `yield` keyword.

During this stage, you must also clean the data before passing it to the second stage. Extract all relevant data found in the row. For the given example, these would be: (`name`, `id`, `address`, `grades`, `communication`: (`phone`, `email`)). To make the next stage easier, it is recommended to save each row as a collection or an object. If a row is missing `id`, `name`, or at least one communication method, the row should be rejected and not passed to the further part of the program.

Additionally, the generator function must be wrapped in a **decorator**. Its task is to count how many rows were read from the file and how many were successfully mapped. After reading is complete, the decorator should print a short summary in the console.

Stage 2: Processing Pipeline

All data read and collected in the first stage enters the pipeline. The order of operations on the provided elements, as well as final actions, is defined by the aforementioned configuration file.

Row Filters (operating on a single collection or object)

Each element fetched from the stage 1 generator should be passed through a chain of connected filters, dynamically built based on the configuration file. The order of filters can be **arbitrary**, and furthermore, not all filters must appear in the configuration (you can omit certain filters).

Elements failing a specific filter, apart from being removed from the chain, should be saved in the `Quarantine` – a dictionary structure where the key is the reason for rejection (information on why the given element was rejected), and the value is a list of all elements/objects that failed this criterion. After all filtering operations are completed, the data from the `Quarantine` should be saved to a `.json` file.

Some filters also include repair logic for elements that fail the filter's criteria. The invocation of repair logic is defined by an appropriate marker in the configuration file (e.g., `[FIX]` or `"repair": true`). If an element fails the filter requirements, before rejecting it and moving it to the `Quarantine`, it should be passed through the repair function/logic and then re-verified by the same filter. A subsequent rejection should result in transferring it to the `Quarantine` and moving on to the next element.
Extra points will be awarded for implementing a more advanced logic where rejected elements go to a separate repair chain and then return to the main flow :>

1. **Filter 1: Type Validator (`CHECK_TYPE`)** - Checks whether values in the element have the expected character types or characters within an appropriate range. E.g., whether the phone consists only of numeric characters after removing separators, whether the email address contains exactly one `@` and no illegal characters, etc. A minimum of 3 elements must be checked in this filter. Creativity in the checked elements will be additionally rewarded.
   Repair: Aggressively removes all illegal characters from the string.

2. **Filter 2: Completeness Inspector (`CHECK_REQUIRED`)** - Based on arguments passed in the configuration file (e.g., `id`, `email`), checks for their presence in the element and ensures their values are not empty (or marked as `None`, `NAN`, or `NO DATA`).
   No repair function.

3. **Filter 3: Normalizer (`NORMALIZE`)** - Cleans data in the following way:
   - **email**: Converts all letters to lowercase and removes whitespaces.
   - **name**: Formats the first and last name so they properly start with capital letters, correctly handling double-barrelled surnames.
   - **phone**: If the phone number has a country code, it should be wrapped in `()`. Additionally, if part of the characters in the phone number were written in a different alphabet variant, all characters should be converted to their "European standard" equivalent (e.g., `٢١٣٧` -> `2137`).
   - **grades**:
     - If passed as a string: Check if some characters were inserted as subscripts (e.g., (\u8324) ₄) and, if necessary, replace them with their "normal" equivalents. Remove `+` or `-` signs preceding some grades.
     - Regardless of format: Truncate the decimal expansion to a maximum of 1 decimal place.

   Repair: In case of a missing first or last name, assigns the value `NO DATA`.

4. **Filter 4: Grade Mapper (`GRADE_MAPPER`)** - Splits a string of grades (e.g., `4.5A,3.0B,5.0A,4.0`) into individual values and removes all incorrect characters from them, including subscripts, special characters, letters, etc. It may happen that after removing the "noise," no character representing a grade remains – in this case, substitute the value `NAN` and pass the row to the `Quarantine`. If all values after mapping are correct, convert them to floats and group them into a list.
   Repair: If, after removing noise and transformations, there is exactly one `NaN` value among the grades, the function should replace this occurrence with the average of the remaining numbers. Otherwise, the row should be rejected.

5. **Filter 5: Memory Guardian (`DEDUPLICATE`)** - Catches duplicates and rejects the element deemed a repetition. An element is considered a duplicate if the same `id` or identical first and last name has already appeared during the program's execution.
   (No repair function).

6. **Filter 6: Checksum Validator (`CHECKSUM`)** - Calculates a weighted sum of the core digits from the `id` number (each digit multiplied by its 1-based index). The result of the modulo 10 operation on this sum must equal the last digit of the `id` (the control digit).

   For REQ-205725, the core is 205725, and the control digit is 5. Sum: $(2 \cdot 1) + (0 \cdot 2) + (5 \cdot 3) + (7 \cdot 4) + (2 \cdot 5) = 55$; $55 \bmod 10 = 5$, which matches the control digit.
   Repair: Reverses the core of the `id` (i.e., indexes backwards), leaving the control digit unchanged, and then recalculates the sum.

Final Operations (operating on all collected data)
Final options are allowed exclusively as the last elements of the processing chain. If a filter is provided in the configuration after one of the final operations, it should be ignored.

1. **Sorting (`SORT`)** - The function sorts elements based on provided attributes. In addition to the attributes contained within the element, the function should allow sorting using: average grade, minimum grade, maximum grade, or the length of the first and last name. Moreover, information about the sort direction (`asc` or `desc`) can be added to each attribute. The order in which attributes are provided should dictate the attribute's sorting priority. If no attributes are provided, the operation should be skipped.
2. **Export (`EXPORT_JSON`)** - Finally, formats the sorted list and saves it to a file named `s#####_report.json` (where `#####` represents the student number).

Stage 3: Data Analysis and Reporting

Using the `pandas` library, load the previously cleaned and sorted data from the `.json` file to conduct the analyses below and present the results.

1. **Console Table:** Data for all students should be divided based on their average grade into three bins/categories: `"At-Risk": < 3.0, "Average": 3.0-4.0, "Top Students": > 4.0`. Next, prepare a table for each of these groups. Within each table, students should be grouped by their e-mail domain (e.g., pejot.edu.pl, uw.edu.pl, pjwstk.edu.pl). The table should contain 3 statistics for each domain: average grade, maximum grade, and the total number of students. Table formatting is flexible and up to the student. The only requirements are that the table must adapt to the size of the data in each column and must have clear headers.
2. **Visualization Panel:** The panel consists of three charts in one main program window. Each graph should be titled, have axis labels, ticks, or data labels in such a way that the represented data is readable to a user unfamiliar with the program.
    - **Bar chart:** Shows the TOP 10 students with the highest average. The color of the bar depends on potential "stumbles": if the student's lowest grade in the semester is 3.5 or higher, the bar should be green; otherwise, it should be red. Labels (first and last name) must appear under the bars, tilted at 45 degrees.
    - **Pie chart:** Represents the percentage share of specific e-mail domains in the entire dataset. The slice representing the most popular domain must be visually exploded (pulled out). Additionally, each chart slice should have its own unique color.
    - **Scatter plot:** Shows the relationship between a student's grade variance and their average grade. The size of each dot on the chart must be proportional to the sum of that student's grades.

Organizational Information and Grading Rules

Passing and Defense

The project defense aims not only to verify the independence of the student's work but also to check the level of understanding of the tools used. Therefore, the student may be questioned on every element used. Issues often arise when a student modifies code via trial and error without understanding the impact of those changes on the program. Because of this, it may happen that an independently completed project is not successfully defended.

```
Failing the defense or discovering a lack of independent work will result
in 0 points for the project.
```

Implementation Notes

The use of external libraries (i.e., those requiring installation via pip) without prior consultation with the instructor is prohibited. The exceptions to this rule are:

- `numpy`
- `pandas`
- `matplotlib`

Submitting Solutions

The only way to submit the project is by attaching it to the appropriate assignment in Microsoft Teams and **clicking** the **Submit** button. It is highly recommended not to submit assignments at the very last minute before the deadline due to potential and unforeseen technical problems.

```
Assignments submitted in any other way will not be graded.
```

As part of the solution, you must submit a single `.zip` archive (containing **only** the `.py` source files and a sample configuration file) named as follows: **`2526S_XY_s#####_ZJM`** where:

- **`XY`** stands for the group number, e.g., 11 or 26 (without the 'c' suffix)
- **`s#####`** stands for the student number, e.g., s23016

The attached configuration file must demonstrate the operation of **all** filters and final operations in such a way that the saved `.json` file can be used to **present** the functionality from **stage three**.

All other project elements such as: environment configuration folder (`.idea`), virtual environment folder (`.venv`), or version control folder (`.git`) must be **removed** before zipping.

```
Failure to comply with any of the above criteria will result in a 5-point
deduction from the project grade.
```

Point Distribution

A maximum of 40 points can be obtained for the project. Points are awarded for the correct operation of individual elements, according to the table below. In the case of errors or partial implementation, partial points will be awarded (with 1-point precision).

Additionally, during the defense:

- If an unhandled error occurs, an additional 5 points are deducted.
- If the project fails to run, an additional 10 points are deducted (the only exception being the need to modify file paths).

Stage	Element	Points
1 (8 pts)	Correct use of generator (yield) and initial extraction	4
	Correct decorator counting statistics	4
2 (20 pts)	Dynamic chain building based on configuration	4
	Implementation of filters: 1, 2, 4, 5, 6 (1 pt for each)	5
	Advanced implementation of filter 3 (Normalizer)	2
	Correct flow of repair functions (re-verification)	3
	Quarantine handling (saving rejected elements with reason)	2
	Dynamic multi-criteria sorting (attributes and ASC/DESC directions)	3
	Saving to JSON file with proper name	1
3 (12 pts)	Multi-level Console Table	5
	Bar chart (conditional colors, slanted labels)	3
	Pie chart (exploded most popular domain, unique colors)	2
	Scatter plot (proper axes and dot size dependency)	2